

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 8
«Алгоритмы на графах»

Выполнил работу

Баранов Владимир

Академическая группа № группы

Принято

Должность, звание Фамилия Имя преподавателя

Санкт-Петербург

2024

ВВЕДЕНИЕ

Цель данной работы — разработка и реализация алгоритмов поиска кратчайшего пути в графе, а также создание инструментов для их тестирования и анализа. Работа включает использование как классических алгоритмов, таких как BFS, DFS, и Дейкстра, так и эвристического подхода, представленного алгоритмом A*.

Задачи:

- Разработать структуру данных для представления графа, включающую узлы и рёбра с весами.
- Реализовать алгоритмы поиска кратчайшего пути:
 - Поиск в ширину (BFS).
 - Поиск в глубину (DFS).
 - Алгоритм Дейкстры.
 - Алгоритм A* с использованием эвристики.
- Разработать механизм загрузки графа из файла в формате списков смежности.
- Создать тестовый набор данных для проверки корректности работы алгоритмов.
- Сравнить результаты работы алгоритмов, выявить их преимущества и недостатки в различных ситуациях.

ТЕОРЕТИЧЕСКАЯ ПОДГОТОВКА

Рассмотрим в теории работу каждого алгоритма:

- Поиск в глубину (DFS):

1. DFS обходит граф, начиная с узла-источника, и погружается в глубину, пока это возможно.
2. Алгоритм:
 - 2.2. Использует стек (или рекурсию) для отслеживания текущего пути.
 - 2.3. Проверяет соседей, посещая их до возврата к предыдущим узлам.
3. DFS не гарантирует нахождение кратчайшего пути, но может быть полезен для анализа графа.

- Поиск в ширину (BFS):

BFS используется для поиска кратчайшего пути в графах

1. Алгоритм:
 - 2.2. Начинается с узла-источника.
 - 2.3. Использует очередь для последовательного обхода соседей.
 - 2.4. Проверяет каждого соседа и сохраняет расстояние до него.
3. BFS гарантирует нахождение кратчайшего пути в графах.

- Алгоритм Дейкстры:

Это классический алгоритм для поиска кратчайшего пути в графах с положительными весами рёбер.

Алгоритм:

1. Инициализирует расстояние до всех узлов как бесконечное, кроме узла-источника.
2. Использует приоритетную очередь для выбора узла с минимальным текущим расстоянием.
3. Обновляет расстояния до соседей текущего узла.
4. Завершается, когда достигнут целевой узел или обработаны все узлы.

5. Гарантирует нахождение кратчайшего пути при условии, что все веса рёбер положительные.

- Алгоритм A*:

A* (A-star) — это модификация алгоритма Дейкстры, которая использует эвристику для ускорения поиска.

Алгоритм:

1. Вычисляет стоимость пути как сумму текущего расстояния от источника и эвристической оценки расстояния до цели.
2. Использует приоритетную очередь для выбора узла с наименьшей оценочной стоимостью.
3. A* эффективен, если эвристика хорошо приближает реальное расстояние до цели, например, эвклидово расстояние в географическом графе.

РЕАЛИЗАЦИЯ

Был создан класс Node, который описывает вершину графа. Вершина содержит:

- Уникальный идентификатор (node_id).
- Координаты (lon, lat).
- Список соседей с указанием веса рёбер.

1. Класс Graph включает методы для управления узлами и рёбрами:

- Добавление узлов и рёбер.
- Хранение узлов в виде списка и кэша для быстрого доступа.
- Загрузка графа из файла в формате списков смежности.

Особенности реализации:

- Использован std::vector для хранения узлов.
- Для быстрого доступа к узлам по координатам реализован std::unordered_map.

2. Реализация метода parseFromFile:

- Данные загружаются из файла, где каждая строка соответствует одной вершине и её рёбрам.
- Считываются координаты вершины и её соседей, создаются соответствующие узлы и рёбра.

Особенности реализации:

- Используются std::stringstream и std::unordered_map для обработки строки и хранения узлов.
- Для кэширования узлов применяется метод createKey, генерирующий уникальный ключ на основе координат.

3. Реализация алгоритмов поиска пути

Алгоритмы поиска:

Реализованы 4 алгоритма:

- BFS — поиск кратчайшего пути в графах с равными весами рёбер.
- DFS — обход графа в глубину.
- Дейкстра — поиск кратчайшего пути в графах с положительными весами.
- A^* — модификация Дейкстры с использованием эвристики.

Особенности реализации:

- В алгоритмах используется структура данных:
- Очередь (`std::queue`) для BFS.
- Стек (`std::stack`) для DFS.
- Приоритетная очередь (`std::priority_queue`) для Дейкстры и A^* .
- В каждом алгоритме реализовано восстановление пути для отладки.

ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

Было проведено два теста: от дома (м. Парнас) до корпуса ИТМО Кронверский 49, другой на специально созданном графе, который без проблем можно посчитать руками, для проверки работы алгоритма.

```
0.0,0.0:1.0,0.0,1.0;0.0,1.0,1.0  
1.0,0.0:1.0,1.0,1.0;2.0,0.0,2.0  
0.0,1.0:1.0,1.0,1.0;0.0,2.0,2.0  
1.0,1.0:2.0,1.0,1.0;1.0,2.0,2.0
```

Изображение №1, тестовый граф.

```
DFS Path: 0 2 3  
DFS Path Length: 2  
DFS Time: 0.000178291 seconds  
BFS Path: 0 1 3  
BFS Path Length: 2  
BFS Time: 3.8166e-05 seconds  
Dijkstra Path: 0 1 3  
Dijkstra Path Length: 2  
Dijkstra Time: 4.0333e-05 seconds  
A* Path: 0 1 3  
A* Path Length: 2  
A* Time: 5.0875e-05 seconds
```

Изображение №2, вывод алгоритма на тестовом графе.

Анализ результатов на тестовом графе:

- Пути:

- DFS нашёл путь $0 \rightarrow 2 \rightarrow 3$, который также имеет длину 2, но отличается от кратчайшего пути, найденного другими алгоритмами.
- BFS, Дейкстра и A* нашли одинаковый путь ($0 \rightarrow 1 \rightarrow 3$), который является кратчайшим.

- Время выполнения:

- BFS оказался самым быстрым алгоритмом (0.000038 сек.).

- A* был самым медленным из трёх алгоритмов для небольшого графа (0.000051 сек.), что связано с дополнительными вычислениями эвристики.
- DFS показал худшее время (0.000178 сек.), так как его стратегия обхода менее оптимальна для поиска кратчайшего пути.

Вывод:

- Для маленького графа, где путь легко найти, BFS, Дейкстра и A* показали одинаковый результат. Однако BFS наиболее эффективен из-за меньших вычислительных затрат.
- DFS не гарантирует нахождение кратчайшего пути и работает дольше.

Анализ результатов на графе г. Санкт-Петербург:

Длина пути:

- DFS нашёл путь с длиной 26.6974, который значительно больше, чем результат других алгоритмов. Это связано с его неэффективной стратегией обхода.
- BFS, Дейкстра и A* нашли кратчайший путь, причём результаты Дейкстры и A* совпали, так как оба гарантируют нахождение оптимального пути.

Время выполнения:

- BFS снова показал хорошее время (0.147654 сек.), но близко к времени Дейкстры (0.146268 сек.).
- A* оказался самым медленным (0.239759 сек.), что связано с вычислением эвристики для большого графа.
- DFS показал значительно большее время (0.383736 сек.) из-за неоптимального обхода и отсутствия приоритетности.

Вывод:

- Для большого графа BFS и Дейкстра оказались наиболее эффективными по времени, причём Дейкстра предоставляет более точные результаты для взвешенного графа.
- A* обеспечивает ту же точность, что и Дейкстра, но из-за вычислений эвристики работает медленнее.
- DFS показал худший результат как по длине пути, так и по времени.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были реализованы и протестированы алгоритмы поиска кратчайшего пути в графах: поиск в глубину (DFS), поиск в ширину (BFS), алгоритм Дейкстры и алгоритм A^* . Цель работы — разработка алгоритмов и их тестирование на графах с различной структурой — была достигнута. BFS, Дейкстра и A^* находят кратчайшие пути, в то время как DFS может находить произвольные пути, что подтверждает его неспособность гарантировать оптимальность. BFS показал лучшие результаты на графах с равными весами рёбер, а Дейкстра и A^* обеспечили наилучшие результаты на взвешенных графах. DFS оказался менее эффективным как по времени выполнения, так и по качеству найденного пути, что делает его неподходящим для задач поиска кратчайшего пути.